

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 7

Processes, Signals, and Pipelines

Course: Operating Systems (Track A: Linux)

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 Core Systems Architecture Concepts 1

2 Part 1: Tracing Execution Mechanics with `strace` **3**
2.1 Tracing a Pipelined Command 3
2.2 System Call Profiling of `ls` 3
2.3 Answers to Questions (Part 1) 4

3 Part 2: Process Creation, Zombies, Quiz, and I/O Buffering **5**
3.1 Basic Fork and Status Retrieval (`fork.c`) 5
3.2 Observing Zombie Processes (`zombie.c`) 5
3.3 Process Tree Doubling Quiz (`quiz.c`) 6
3.4 I/O Buffering Mechanics (`./fork` vs. `./fork | cat`) 7
3.5 Answers to Questions (Part 2) 7

4 Part 3: Program Execution and Signal Handling (`myrun.c`) **9**
4.1 Implementation of `myrun.c` 9
4.2 Execution Traces and Verification 9
4.3 POSIX Status Decoding Mechanics 10

5 Part 4: File Descriptor Redirection (`redirect.c` & `myrun2.c`) **11**
5.1 Basic Redirection via `dup2` (`redirect.c`) 11
5.2 Generalized Command Runner with File Output (`myrun2.c`) 11
5.3 File Descriptor Table Transformation 12

6 Part 5: Inter-Process Communication with Pipelines **14**
6.1 Two-Stage Pipeline (`pipeline.c`) 14
6.2 Three-Stage Pipeline (`pipeline3.c`) 15
6.3 Deadlock Prevention and EOF Semantics 16

7 Conclusions **17**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

In UNIX and modern Linux operating systems, the process abstraction is the primary mechanism for computational isolation and concurrency. The primary objective of Laboratory Work Nr. 7 is to explore the Linux process model, process execution lifecycle, standard stream redirection, and inter-process communication (IPC) via pipes.

Throughout this laboratory, we investigate:

- **Tracing Process Management Syscalls (`strace`)**: Observing how the shell orchestrates pipelines via anonymous pipes (`pipe2`), process replication (`clone`), file descriptor reassignment (`dup2`), and executable loading (`execve`).
- **Process Lifecycle, Concurrency, and Reaping**: Utilizing `fork()` and `waitpid()` to spawn concurrent child processes, monitor execution states, and harvest exit statuses using POSIX macros (`WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`).
- **Zombie and Orphan Management**: Observing zombie processes in state `Z` (`<defunct>`) using `ps`, examining the responsibilities of parent processes in status harvesting, and understanding reparenting to PID 1.
- **Exponential Process Trees**: Analyzing process duplication across sequential `fork()` invocations and verifying the resulting 2^n process tree topology.
- **I/O Buffering and Address Space Duplication**: Analyzing the critical difference between line-buffered interactive terminals (TTY) and block-buffered pipes/files, explaining duplicate prints from un-flushed libc buffers across `fork()`.
- **Program Execution and Signal Trapping (`myrun`)**: Building a wrapper utility around `execvp` to execute binaries, pass arguments, handle execution failures, and detect termination by asynchronous signals (e.g., `SIGTERM`).
- **Stream Redirection (`redirect` and `myrun2`)**: Manipulating per-process open file descriptor tables using `dup2()` to route standard output (`STDOUT_FILENO`) to regular filesystem files.
- **Inter-Process Communication Pipelines (`pipeline` and `pipeline3`)**: Developing two-stage and three-stage data processing pipelines using anonymous pipes, coordinating data flow between stages, and avoiding deadlocks through disciplined file descriptor closing.

1.2 Core Systems Architecture Concepts

1. **The Fork-Exec-Wait Paradigm**: Unlike operating systems that spawn processes via a single unified call (such as `CreateProcess` on Windows), UNIX decouples process creation from executable loading. `fork()` duplicates the calling process's entire virtual address space, memory pages (via Copy-On-Write), and file descriptor table. The child process can subsequently modify its environment and descriptor table before calling `execve()`, which replaces the process memory image with a new binary.

2. **File Descriptors and the Open File Table**: Every process has a file descriptor table. Standard slots 0, 1, and 2 correspond to `stdin`, `stdout`, and `stderr`. The system call `dup2(oldfd, newfd)` atomically closes `newfd` (if open) and duplicates `oldfd` into slot `newfd`, sharing the underlying open file description and file offset pointer.

3. **Anonymous Pipes**: An anonymous pipe is a unidirectional FIFO byte stream allocated in kernel memory with a designated read end (`p[0]`) and write end (`p[1]`). Readers block when data is temporarily unavailable. Crucially, a reader receives `EOF` (0 bytes read) if and only if **every** open file descriptor pointing to the write end of the pipe across all processes in the system has been closed.

2 Part 1: Tracing Execution Mechanics with `strace`

2.1 Tracing a Pipelined Command

We ran `strace` with the `-f` flag (to follow child processes) and filtered for process management and I/O syscalls while executing the pipeline `ls / | wc -l` in GNU Bash:

```
$ strace -f -e trace=pipe2,clone,clone3,dup2,execve,wait4 bash -c 'ls / | wc -l'
execve("/usr/bin/bash", ["bash", "-c", "ls / | wc -l"], 0x7ffd1ada4d18 /* 199 vars
    */) = 0
pipe2([3, 4], 0)                        = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7eb6a3a42e50) = 15021
strace: Process 15021 attached
[pid 15020] clone(child_stack=NULL,
    flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7eb6a3a42e50) = 15022
[pid 15021] dup2(4, 1)                  = 1
strace: Process 15022 attached
[pid 15020] wait4(-1 <unfinished ...>
[pid 15022] dup2(3, 0)                  = 0
[pid 15021] execve("/usr/bin/ls", ["ls", "/"], 0x5bf3950f21b0 /* 199 vars */
    <unfinished ...>
[pid 15022] execve("/usr/bin/wc", ["wc", "-l"], 0x5bf3950f21b0 /* 199 vars */
    <unfinished ...>
[pid 15021] <... execve resumed>)       = 0
[pid 15021] +++ exited with 0 +++
[pid 15020] <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
    15021
[pid 15020] wait4(-1 <unfinished ...>
[pid 15022] <... execve resumed>)       = 0
19
[pid 15022] +++ exited with 0 +++
<... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 15022
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=15021, si_uid=1000,
    si_status=0, si_utime=0, si_stime=0} ---
wait4(-1, 0x7ffeff115694, WNOHANG, NULL) = -1 ECHILD (No child processes)
+++ exited with 0 +++
```

2.2 System Call Profiling of `ls`

We profiled the system calls issued by `ls` when reading the directory `/usr/bin`:

```
$ strace -c ls /usr/bin > /dev/null
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 56.20    0.000843         140         6           getdents64
 25.73    0.000386         386         1           execve
  7.67    0.000115           8        14           mmap
  2.60    0.000039           9         4           mremap
  1.53    0.000023           4         5           openat
  1.47    0.000022           3         6           brk
  1.27    0.000019           6         3           newfstatat
  1.13    0.000017           2         6           fstat
  0.73    0.000011           1         7           close
  0.67    0.000010           0        12           write
  0.60    0.000009           4         2           read
```

```
  0.33    0.000005           5         1         1 access
  0.07    0.000001           0         2         2 ioctl
------ ----------- ----------- --------- --------- ----------------
100.00    0.001500          17        87         7 total
```

2.3 Answers to Questions (Part 1)

(1) In what order do `pipe2`, `clone`, `dup2` and `execve` appear? Which process (PID) calls `dup2(.., 1)` and which calls `dup2(.., 0)`? Explain why in one sentence each.

- **Syscall Order**:

  `pipe2` $\longrightarrow$ `clone` (child 1) $\longrightarrow$ `clone` (child 2)
  $\longrightarrow$ `dup2` (in each child) $\longrightarrow$ `execve` (in each child)

  The parent bash process first creates the communication channel with `pipe2([3, 4], 0)`, then clones both child processes, after which each child binds its standard streams via `dup2` before finally executing its respective binary via `execve`.

- **`dup2(4, 1)` Caller**: Process **PID 15021** (`ls`) calls `dup2(4, 1)` to bind its standard output (`fd 1`) to the write end of the pipe (`fd 4`) so that directory listing data is piped forward.

- **`dup2(3, 0)` Caller**: Process **PID 15022** (`wc`) calls `dup2(3, 0)` to bind its standard input (`fd 0`) to the read end of the pipe (`fd 3`) so that line counting consumes the data streamed through the pipe.

(2) On Linux there is no "fork" line. Which call creates the children instead?

On Linux, the **`clone`** (or **`clone3`**) system call creates the child processes instead of a traditional `fork` syscall. In the GNU C Library (glibc), `fork()` is implemented as a specialized invocation of:

```
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID | CLONE_CHILD_SETTID |
                              SIGCHLD, ...)
```

The `clone` syscall offers a unified, extensible interface that underpins both process creation and POSIX thread instantiation (`pthreads`) by parametrically controlling which virtual memory, file descriptor tables, and namespaces are copied or shared.

(3) How many write calls does `ls` make to print all of `/usr/bin`? Why so few?

`ls` issued only **12 `write` calls** to output several thousand binary names. This efficiency is achieved through **user-space stream buffering** implemented by the C standard library (`libc`). Rather than issuing an individual kernel-crossing `write()` system call for each individual file string, formatted output is accumulated inside an internal user-space buffer (typically 4096 or 8192 bytes) and flushed in large, contiguous blocks, minimizing context switches between user and kernel mode.

3 Part 2: Process Creation, Zombies, Quiz, and I/O Buffering

3.1 Basic Fork and Status Retrieval (`fork.c`)

We created and compiled `fork.c` to inspect process IDs and exit status harvesting:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("before fork: pid=%d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) { /* child */
        printf("child: pid=%d parent=%d\n", getpid(), getppid());
        exit(42);
    }

    int status; /* parent */
    waitpid(pid, &status, 0);
    printf("parent: pid=%d, my child %d exited with %d\n",
           getpid(), pid, WEXITSTATUS(status));
    return 0;
}
```

Execution trace:

```
$ gcc -Wall -Wextra -o fork fork.c && ./fork
before fork: pid=16968
child: pid=16969 parent=16968
parent: pid=16968, my child 16969 exited with 42
```

3.2 Observing Zombie Processes (`zombie.c`)

To observe the zombie process state, we developed `zombie.c`, where the child terminates immediately while the parent sleeps without calling `wait()`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
```

```
        exit(1);
    }

    if (pid == 0) {
        printf("Child (PID %d) is exiting now...\n", getpid());
        exit(0);
    } else {
        printf("Parent (PID %d) created child (PID %d).\n", getpid(), pid);
        printf("Parent sleeping for 30 seconds. Check `ps` in another terminal!\n");
        sleep(30);
        printf("Parent waking up and exiting.\n");
    }

    return 0;
}
```

We launched the program in the background and queried the process table:

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie &
[2] 17428
Parent (PID 17428) created child (PID 17430).
Parent sleeping for 30 seconds. Check `ps` in another terminal!
Child (PID 17430) is exiting now...

$ sleep 1; ps -o pid,ppid,stat,cmd --ppid $!
    PID    PPID STAT CMD
  17430   17428 Z    [zombie] <defunct>

Parent waking up and exiting.
```

3.3 Process Tree Doubling Quiz (`quiz.c`)

We implemented `quiz.c` containing three consecutive `fork()` invocations:

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    fork();
    fork();
    fork();
    printf("hi\n");
    return 0;
}
```

Execution and output line count:

```
$ gcc -Wall -o quiz quiz.c && ./quiz | wc -l
8
```

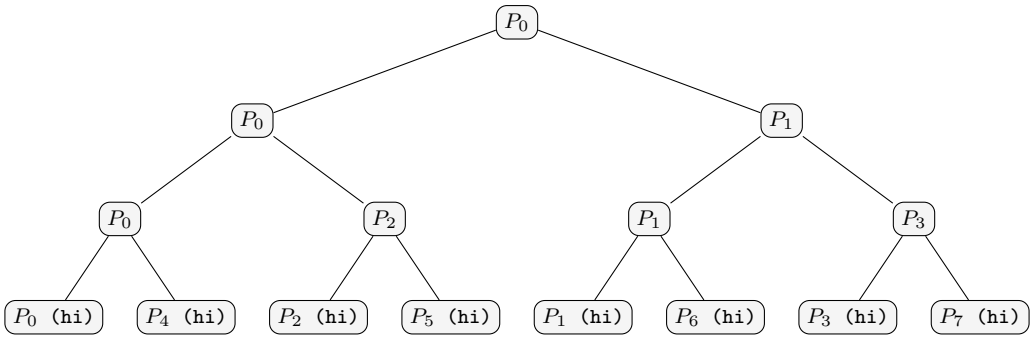


Figure 1: Process Tree generated by three successive `fork()` calls in `quiz.c` resulting in 8 distinct processes.

3.4 I/O Buffering Mechanics (`./fork` vs. `./fork | cat`)

When `fflush(stdout)` was commented out in `fork.c`, we observed differing behavior between terminal execution and pipe redirection:

```
$ ./fork
before fork: pid=18134
child: pid=18135 parent=18134
parent: pid=18134, my child 18135 exited with 42

$ ./fork | cat
before fork: pid=18198
child: pid=18200 parent=18198
before fork: pid=18198
parent: pid=18198, my child 18200 exited with 42
```

3.5 Answers to Questions (Part 2)

(1) What does the STAT column show for the child while the parent sleeps, and what happens to it after the parent calls `waitpid`?

- **While Parent Sleeps**: The `STAT` column displays `Z` (`[zombie] <defunct>`). The child process has completed its execution, released its virtual address space and file descriptors, but its entry in the kernel process table (`task_struct`) is preserved so that its termination status code can be retrieved.
- **After `waitpid` (or Parent Exit)**: When the parent calls `waitpid()`, the exit status is acknowledged and the kernel frees the child's entry from the process table, reclaiming its PID. If the parent terminates without calling `waitpid()`, the zombie child is reparented to `init` (PID 1), which automatically harvests it.

(2) Your prediction for `quiz.c` and the real count. Draw the process tree (8 boxes).

- **Prediction vs. Real Count**: The real count verified by `./quiz | wc -l` is **8**.
- **Process Tree Analysis**: Each `fork()` invocation doubles the number of active processes:

$$1 \xrightarrow{\text{fork 1}} 2 \xrightarrow{\text{fork 2}} 4 \xrightarrow{\text{fork 3}} 8 \text{ processes}$$

Figure 1 shows the 8 distinct execution leaves, each executing `printf("hi\n")`.

(3) Without `fflush`, why does "before fork" appear twice only when the output goes into a pipe?

The C standard library (`libc`) implements two distinct buffering modes for `stdout`:

1. **Line Buffering (Terminal)**: When connected to an interactive TTY, `stdout` flushes immediately upon encountering a newline character (`\n`). Therefore, the `write()` system call is executed before `fork()` runs, leaving the user-space buffer empty.

2. **Full Block Buffering (Pipe / File)**: When connected to a pipe or file, `stdout` switches to block buffering (typically 4096 bytes). The string `"before fork: pid=...\n"` remains stored in the parent's user-space memory buffer. When `fork()` is called, the kernel duplicates the parent's entire address space via Copy-On-Write, **including the unflushed libc buffer**. When the parent and child eventually terminate, both processes flush their private buffer copies, causing the string to be written into the pipe twice.

4 Part 3: Program Execution and Signal Handling (`myrun.c`)

4.1 Implementation of `myrun.c`

We implemented the command runner `myrun.c` using `execvp` to search `$PATH`, and evaluated the process termination status using POSIX macros:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* usage: ./myrun program [args...] e.g. ./myrun ls -l /tmp */
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s program [args...]\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Replace process with target binary, searching PATH */
        execvp(argv[1], &argv[1]);

        /* If execvp returns, execution failed */
        perror("execvp");
        exit(127);
    }

    int status;
    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }

    /* Inspect termination status */
    if (WIFEXITED(status)) {
        printf("[myrun] %s exited with status %d\n", argv[1], WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("[myrun] %s killed by signal %d\n", argv[1], WTERMSIG(status));
    }

    return 0;
}
```

4.2 Execution Traces and Verification

We verified `myrun` across four distinct process termination conditions:

```
$ ./myrun ls -d /etc
/etc
[myrun] ls exited with status 0

$ ./myrun false
[myrun] false exited with status 1

$ ./myrun nosuchprog
execvp: No such file or directory
[myrun] nosuchprog exited with status 127

$ ./myrun sleep 100 & sleep 1; pkill -x sleep
[1] 20105
[myrun] sleep killed by signal 15
```

4.3 POSIX Status Decoding Mechanics

The integer populated by `waitpid` encodes both normal and signal-induced termination information:

- `WIFEXITED(status)` evaluates to true if the process terminated via `exit()` or return from `main()`. The exit code is extracted using `WEXITSTATUS(status)` (bits 8–15 of the status word).
- `WIFSIGNALED(status)` evaluates to true if the child was terminated by an unhandled signal (such as `SIGTERM` / signal 15 sent by `pkill`). The terminating signal number is extracted using `WTERMSIG(status)`.

5 Part 4: File Descriptor Redirection (`redirect.c` & `myrun2.c`)

5.1 Basic Redirection via `dup2` (`redirect.c`)

We constructed `redirect.c` to demonstrate stream redirection at the file descriptor table level:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        /* Child: redirect stdout (fd 1) to out.txt */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }

        dup2(fd, 1);    /* stdout now points to out.txt */
        close(fd);      /* close original descriptor */

        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp");
        exit(127);
    }

    /* Parent */
    int status;
    waitpid(pid, &status, 0);
    return 0;
}
```

Verification and file output check:

```
$ ./redirect && cat out.txt | head -3
total 120
-rwxr-xr-x 1 sunflower sunflower 16328 Oct 18 18:12 fork
-rw-r--r-- 1 sunflower sunflower  3834 Oct 18 18:12 fork.c
```

5.2 Generalized Command Runner with File Output (`myrun2.c`)

We implemented `myrun2.c` to redirect stdout to an arbitrary target file specified on the command line:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

/* usage: ./myrun2 outfile program [args...] */
int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s outfile program [args...]\n", argv[0]);
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }
        dup2(fd, STDOUT_FILENO);
        close(fd);

        execvp(argv[2], &argv[2]);
        perror("execvp");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("[myrun2] %s exited with status %d (output written to %s)\n",
               argv[2], WEXITSTATUS(status), argv[1]);
    } else if (WIFSIGNALED(status)) {
        printf("[myrun2] %s killed by signal %d\n", argv[2], WTERMSIG(status));
    }

    return 0;
}
```

Verification trace:

```
$ ./myrun2 out.txt ls -l /etc
[myrun2] ls exited with status 0 (output written to out.txt)
```

5.3 File Descriptor Table Transformation

Table 1 depicts the evolution of the child process's file descriptor table during redirection:

Step	**Descriptor 1 (`stdout`)**	**Descriptor 3 (`fd`)**
1. Initial State (after `fork`)	Terminal Device (`/dev/pts/...`)	Unused
2. After `open("out.txt", ...)`	Terminal Device (`/dev/pts/...`)	Regular File (`out.txt`)
3. After `dup2(3, 1)`	Regular File (`out.txt`)	Regular File (`out.txt`)
4. After `close(3)`	Regular File (`out.txt`)	Closed / Unused
5. After `execvp`	Regular File (`out.txt`) [Preserved]	Closed / Unused

Table 1: Transformation of child process file descriptor table across redirection steps.

6 Part 5: Inter-Process Communication with Pipelines

6.1 Two-Stage Pipeline (`pipeline.c`)

We implemented `pipeline.c` to execute `ls -l /etc | wc -l` using an anonymous pipe:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) {
        perror("pipe");
        exit(1);
    }

    /* Child 1: ls -l /etc */
    pid_t pid1 = fork();
    if (pid1 < 0) {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0) {
        close(p[0]);               /* not reading */
        dup2(p[1], STDOUT_FILENO); /* redirect stdout to pipe write end */
        close(p[1]);

        execlp("ls", "ls", "-l", "/etc", (char *)NULL);
        perror("execlp ls");
        exit(127);
    }

    /* Child 2: wc -l */
    pid_t pid2 = fork();
    if (pid2 < 0) {
        perror("fork");
        exit(1);
    }

    if (pid2 == 0) {
        close(p[1]);               /* not writing */
        dup2(p[0], STDIN_FILENO); /* redirect stdin to pipe read end */
        close(p[0]);

        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp wc");
        exit(127);
    }

    /* Parent closes both pipe ends so EOF can be reached */
    close(p[0]);
    close(p[1]);

    /* Wait for both children */
```

```
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

Verification trace:

```
$ ./pipeline
192
```

6.2 Three-Stage Pipeline (`pipeline3.c`)

We implemented a three-stage pipeline executing `ls /usr/bin | grep x | wc -l` utilizing two anonymous pipes:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p1[2], p2[2];
    if (pipe(p1) < 0 || pipe(p2) < 0) {
        perror("pipe");
        exit(1);
    }

    /* Stage 1: ls /usr/bin -> p1[1] */
    if (fork() == 0) {
        dup2(p1[1], STDOUT_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        exit(127);
    }

    /* Stage 2: p1[0] -> grep x -> p2[1] */
    if (fork() == 0) {
        dup2(p1[0], STDIN_FILENO);
        dup2(p2[1], STDOUT_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        exit(127);
    }

    /* Stage 3: p2[0] -> wc -l */
    if (fork() == 0) {
        dup2(p2[0], STDIN_FILENO);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        exit(127);
    }

    /* Parent closes all pipe ends */
```

```
43      close(p1[0]); close(p1[1]);
44      close(p2[0]); close(p2[1]);
45
46      /* Wait for all 3 children */
47      while (wait(NULL) > 0);
48
49      return 0;
50  }
```

Verification against native shell execution:

```
$ ./pipeline3
552

$ \ls /usr/bin | grep x | wc -l
552
```

6.3 Deadlock Prevention and EOF Semantics

The disciplined closure of pipe descriptors is vital for proper pipeline execution:

- When a process reads from an empty pipe, the kernel puts the reading process to sleep until data is written, **unless** all descriptors pointing to the write end of that pipe have been closed.
- When the last write descriptor is closed, subsequent read operations immediately return `0` bytes, signaling **End-Of-File (EOF)**.
- If the parent process or intermediate stages neglect to close their write descriptors (e.g. `p1[1]` or `p2[1]`), the write reference count in the kernel will never drop to zero. The final stage (`wc -l`) will remain permanently blocked waiting for more input, resulting in an unresolved deadlock.

7 Conclusions

Laboratory Work Nr. 7 provided thorough, hands-on experience with UNIX process creation, coordination, file descriptor remapping, and inter-process communication:

1. **Modularity of Fork and Exec**: System call analysis with `strace` highlighted the elegance of decoupling address space creation (`clone`/`fork`) from executable loading (`execve`). This architectural separation enables parent shells to configure process privileges, resource limits, and file descriptor tables before passing control to target binaries.

2. **Process Lifecycle and Zombie Prevention**: Experiments with `zombie.c` proved that processes do not simply vanish upon exiting; their kernel descriptor remains in state `Z` until reaped by their parent via `waitpid()`. Proper status reaping is essential to prevent process table exhaustion in long-running applications.

3. **Impact of Address Space Duplication on User Buffers**: The buffering experiment revealed that user-space libc stream buffers are duplicated during `fork()`. Because interactive terminals use line buffering while pipes use block buffering, unflushed buffers produce duplicate writes unless explicitly handled via `fflush()`.

4. **Status Introspection and Signal Handling**: Building `myrun.c` demonstrated how to decode the POSIX status integer to distinguish between voluntary process termination (`WIFEXITED`, `WEXITSTATUS`) and involuntary signal termination (`WIFSIGNALED`, `WTERMSIG`).

5. **Stream Redirection and Pipeline Synchronization**: Developing `redirect.c`, `myrun2.c`, `pipeline.c`, and `pipeline3.c` demonstrated the power of file descriptor remapping via `dup2`. Successful pipeline execution requires strict descriptor discipline: closing every unused pipe end is necessary to ensure that EOF propagates correctly and prevents deadlocks.